

## JAVA 17 OCP · YDS STUDY SERIES

# Unit 05 Grammar Notes · Methods

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

## 1. which lines ...?

**EN:** Which lines in the method generate a compiler error?

**TR:** Method içindeki hangi satırlar compiler error üretir?

`which + plural noun` seçim sorusu kurar ve plural verb alır.

## 2. if + passive

**EN:** What is printed if the erroneous line is removed?

**TR:** Hatalı satır kaldırılırsa ne yazdırılır?

Teknik olasılıkta `if + present passive`, sonuç clause'uyla kullanılır.

## 3. as if + clause

**EN:** The static method is called as if it were an instance method.

**TR:** `static` method, instance method'muş gibi çağrılır.

`as if + past` gerçek olmayan görünüş/varsayım bildirir.

## 4. provided that

**EN:** A variable is effectively final, provided that it is not reassigned.

**TR:** Yeniden atanmadığı sürece variable effectively final'dır.

Koşul formülü: `main clause + provided (that) + clause`.

## 5. since ile neden

**EN:** The caller is unchanged since Java is pass-by-value.

**TR:** Java pass-by-value olduğu için caller değişmez.

## 6. even though

Beklenmedik karşıtlık bildirir.

**EN:** The call compiles even though the reference is null.

**TR:** Reference null olmasına rağmen çağrı derlenir.

## 7. at most

**EN:** A method may declare at most one varargs parameter.

**TR:** Bir method en fazla bir varargs parameter bildirebilir.

## 8. by the time + clause

**EN:** By the time you finish this chapter, the rules will fit together.

**TR:** Bu bölümü bitirdiğinizde kurallar bir bütün hâline gelmiş olacak.

Gelecekte belirli bir ana kadar tamamlanacak durumu anlatır: `by the time + present simple, future perfect/simple future`.

## 9. while ile karşıtlık

**EN:** Autoboxing converts a primitive to a wrapper, while unboxing does the reverse.

**TR:** Autoboxing primitive'i wrapper'a dönüştürürken unboxing tersini yapar.

YDS'de `while` yalnız “-iken” zaman anlamı taşımaz; iki bilgiyi karşılaştırabilir.

## 10. regardless of whether + clause

**EN:** The value does not change, regardless of whether it is explicitly marked as `final`.

**TR:** Açıkça `final` olarak işaretlenip işaretlenmediğine bakılmaksızın value değişmez.

İki olasılığın da sonucu etkilemediğini gösterir:

`main clause + regardless of whether + subject + verb`

YDS ipucu: `regardless of` sonrasında noun/gerund; `regardless of whether` sonrasında clause gelir.

## 11. rather than

**EN:** The parameters are separated by a semicolon rather than a comma.

**TR:** Parameter'lar virgül yerine noktalı virgülle ayrılmıştır.

Tercih veya karşılaştırma formülü: `X rather than Y` → “Y yerine X”. Paralel yapıyı koruyun: noun–noun, verb–verb veya clause–clause.

## 12. whether + clause ile dolaylı soru

**EN:** The compiler determines whether the call is allowed.

**TR:** Compiler, çağrıya izin verilip verilmediğini belirler.

`whether + subject + verb`, yes/no sorusunu noun clause'a dönüştürür. Türkçede çoğu kez “-ip -mediği” yapısıyla karşılanır. `if` bazı bağlamlarda mümkün olsa da `whether`, iki olasılığı daha açık vurgular.

## 13. Amaç bildiren so (that)

**EN:** We mention it so you don't get confused when practicing.

**TR:** Alıştırma yaparken kafanız karışmasın diye bundan söz ediyoruz.

Formül: `main clause + so (that) + subject + can/could/will/would + verb`.  
Informal technical prose'da modal atlanabilir; buradaki kaynak cümlede `that` da yazılmamıştır.

## 14. Açıklayıcı relative clause: which + verb

**EN:** This is called a method declaration, which specifies all the information needed to call the method.

**TR:** Buna, method'u çağırmak için gereken tüm bilgileri belirten method declaration denir.

Virgülden sonraki `which` clause, önceki noun hakkında ek bilgi verir. Non-defining relative clause olduğu için `that` kullanılmaz ve clause virgülle ayrılır.

## 15. `as many ... as`

**EN:** You can list as many types of exceptions as you want.

**TR:** İstedığınız kadar exception type'ı listeleyebilirsiniz.

Sayılabilen plural noun'larla miktar eşitliği veya üst sınırın bulunmadığını anlatır:

`as many + plural noun + as + clause`.

## 16. `once + clause`

**EN:** Once the return type is specified, the rest of the method follows a specific order.

**TR:** Return type belirtildikten sonra method'un geri kalanı belirli bir sıra izler.

`once`, tamamlanan bir olayı sonuca bağlar; geleceğe gönderme yapsa bile time clause içinde genellikle present tense kullanılır.

## 17. `unless + passive clause`

**EN:** Methods are required to have a body unless they are declared `abstract`.

**TR:** Method'lar `abstract` bildirilmedikçe body'ye sahip olmak zorundadır.

`unless`, “if ... not” anlamındadır:

`main clause + unless + subject + be + past participle`. Aynı clause'a ayrıca `not` eklemek double negative hatasına yol açabilir.

## 18. `It is not until ... that ...`

**EN:** It isn't until the third call that the varargs version is used.

**TR:** Varargs sürümü ancak üçüncü call'da kullanılır.

Bu cleft structure, olayın belirtilen ana kadar gerçekleşmediğini vurgular:

`It is/was not until + time/event + that + clause`; Türkçede çoğu kez “ancak” kullanılır.

## Mini quiz

| Soru                                                                            |
|---------------------------------------------------------------------------------|
| 1. <code>as if</code> ile static method çağrısı hakkında cümle yaz.             |
| 2. <code>provided that</code> kullanarak effectively final kuralını belirt.     |
| 3. <code>even though</code> ile null static call karşıtlığını kur.              |
| 4. <code>at most</code> kalıbını varargs ile kullan.                            |
| 5. <code>unless</code> kullanarak abstract method body kuralını yaz.            |
| 6. <code>rather than</code> kullanarak geçersiz parameter separator örneği kur. |

## Cevaplar

| Cevap                                                                                |
|--------------------------------------------------------------------------------------|
| 1. <i>The static method is called as if it were an instance method.</i>              |
| 2. <i>A local variable is effectively final, provided that it is not reassigned.</i> |
| 3. <i>The static call succeeds even though the reference is null.</i>                |
| 4. <i>A method can have at most one varargs parameter.</i>                           |
| 5. <i>Methods need a body unless they are declared abstract.</i>                     |
| 6. <i>The parameters are separated by a semicolon rather than a comma.</i>           |